

Kubernetes Pod Disruption Budgets (PDB)

What is a Pod Disruption Budget (PDB)?

- A Pod Disruption Budget (PDB) is a Kubernetes policy ensures that a minimum number of pods are always running during voluntary disruptions.
-  Examples of Voluntary Disruptions:
 - Node drain 
 - Cluster upgrades 
 - Rolling updates 
 - Manual pod eviction

Purpose of PDB

-  Ensures High Availability (HA) by controlling how many pods can go down at once.
-  PDB = Safety rule for pods during disruptions

Why PDB (Pod Disruption Budget) is Important

 Scenario	 Behavior	 Impact
 Without PDB	 All pods may go down during updates (killed at once)	 Application downtime
 With PDB	 Controlled disruptions (minimum pods stay running)	 High Availability (HA), zero/low downtime

Voluntary vs Involuntary Disruptions

 Type	 Examples	 PDB Control
 Voluntary Disruptions	 Node drain (<code>kubectl drain</code>)  Rolling updates  Manual pod eviction	 Controlled by PDB
 Involuntary Disruptions	 Node crash  Hardware failure  Kernel panic  Pod crash	 Not controlled by PDB

- 👉 PDB only works for voluntary disruptions



Example 1: minAvailable

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-app-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: my-app
```

- ✅ At least 2 pods must always be running



Example 2: maxUnavailable

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-app-pdb
spec:
  maxUnavailable: 25%
  selector:
    matchLabels:
      app: my-app
```

- ✅ Allows 25% of pods to be unavailable



Important Rule

- 👉 PDB defines how many pods must stay available, You can use ONLY ONE :
 - ✅ minAvailable
 - ✅ maxUnavailable
 - ❌ Not both together
-



Real Scenario

Setup:

- Deployment with 5 pods
- PDB: minAvailable: 4



What happens during node drain?

- Node has 2 pods
- Kubernetes tries to `evict` both
- ❌ PDB allows only 1 eviction
- 🛑 Drain pauses
- New pod starts elsewhere
- Then second pod can be evicted



Key Clarification

- ❌ PDB does NOT control pod placement
- ❌ Does NOT force pods to stay on nodes
- ✅ Only limits disruption count



Without PDB

- ⚠️ Risk:
 - All pods may go down at once
 - Leads to downtime
 - Causes instability



Key Notes

- Works with Deployments , StatefulSets 🌀
- Not guaranteed for crashes ⚠️
- Ensures safe updates ↺
- Critical for zero downtime 🚀



Pod Disruption Budgets (PDB) in Kubernetes — Rapid Fire Q&A

 Q#	? Question	💡 Answer
◆ Q1	What is a Pod Disruption Budget (PDB)?	👉 A policy that limits how many pods can be voluntarily disrupted at a time.
◆ Q2	What is a “voluntary disruption”?	👉 Actions like node drain , rolling updates , or manual pod deletion .
◆ Q3	Does PDB prevent pod crashes?	👉 ❌ No – it only controls voluntary disruptions, not failures.
🧠 Q4	What are the two main PDB fields?	👉 minAvailable and maxUnavailable
🧠 Q5	What is minAvailable?	👉 Minimum number of pods that must remain available.
🧠 Q6	What is maxUnavailable?	👉 Maximum number of pods that can be unavailable .
🧠 Q7	Can we use both together?	👉 ❌ No – only one can be defined.
📄 Q8	Example of PDB YAML?	👉 <pre>apiVersion: policy/v1 kind: PodDisruptionBudget metadata: name: my-app-pdb spec: minAvailable: 2 selector: matchLabels: app: my-app</pre>
⚙️ Q9	What happens if disruption violates PDB?	👉 Kubernetes blocks the eviction .
⚙️ Q10	Who enforces PDB?	👉 Kubernetes API server during eviction requests .
⚙️ Q11	What is eviction?	👉 Graceful pod termination via API.
🌀 Q12	Which workloads commonly use PDB?	👉 Deployments, StatefulSets.
🌀 Q13	Why is PDB important in production?	👉 Ensures high availability during maintenance .
🔧 Q14	What happens during kubectl drain?	👉 Pods are evicted respecting PDB rules.
🔧 Q15	What if PDB blocks drain?	👉 Drain fails until conditions are met.
📊 Q16	Example: 5 pods, minAvailable=3 – how many can be disrupted?	👉 2 pods

 Q#	? Question	💡 Answer
 Q17	Example: 5 pods, maxUnavailable=1 — how many can be down?	👉 Only 1 pod
 Q18	Does PDB guarantee zero downtime?	👉 ❌ No — it only reduces risk.
 Q19	Does PDB affect scaling?	👉 ❌ No — scaling ignores PDB.
 Q20	Does PDB affect rolling updates?	👉 ✅ Yes — limits how many pods can go down.
 Q21	Pod not getting evicted?	👉 Check PDB constraints.
 Q22	Node drain stuck?	👉 Likely blocked by PDB.
 Q23	Too strict PDB impact?	👉 Can prevent deployments or maintenance.
✅ Q24	When to use minAvailable?	👉 When you know minimum required pods.
✅ Q25	When to use maxUnavailable?	👉 When defining acceptable disruption.
✅ Q26	For single replica apps?	👉 Avoid strict PDB (can block all operations).
🎯 Q27	Why did node drain fail in production?	👉 PDB prevented pod eviction.
🎯 Q28	How to fix strict PDB?	👉 Adjust values or scale replicas.
🎯 Q29	What happens if replicas < minAvailable?	👉 No pod can be evicted.
🎯 Q30	How to design HA using PDB?	👉 Combine replicas + PDB + readiness probes .

✨ Summary

PDB helps:

- 🛑 Prevent too many pods going down

-  Enable safe rolling updates
-  Maintain high availability
-  Ensure production stability
-  Essential for reliable Kubernetes deployments 